

(□ι_ι□) Church-encoding
text into iotas

The backstory



Thank You!

A mysterious QR code...

...leading to peculiar runes

(l(l(l(l))))(l(l(l(l))))(l(l(l(l))(l(l(l(l))))))
 (l(l(l(l))))(l(l(l(l))(l(l(l(l)))))(l(l(l(l))(l(l
 (l(l(l(l))(l(l(l(l))))(l(l(l(l))(l(l(l(l))))
 (l(l(l(l)))))(l(l(l(l))(l(l(l(l))(l(l(l(l)))))(l(l))
 (l(l(l(l))(l(l(l(l))))(l(l(l(l))(l(l(l(l))))))
 (l(l(l(l))(l(l(l(l))(l(l(l(l)))))(l(l)))(l(l(l(l))))
 (l(l(l(l))(l(l(l(l))(l(l(l(l)))))))(l(l(l(l))))
 (l(l(l(l))(l(l(l(l)))))(l(l)))(l(l(l(l))(l(l(l(l))))
 (l(l(l(l))(l(l(l(l)))))(l(l(l(l))(l(l(l(l))))
 (l(l(l(l)))))(l(l)))(l(l(l(l))(l(l(l(l))(l(l(l(l))))
 (l(l(l(l)))))(l(l(l(l))(l(l(l(l))(l(l(l(l)))))(l(l))
 (l(l(l(l))(l(l(l(l))(l(l(l(l))(l(l(l(l))))))
 (l(l(l(l))(l(l(l(l))(l(l(l(l)))))(l(l)))(l(l(l(l))))
 (l(l(l(l))(l(l(l(l))(l(l(l(l)))))))(l(l(l(l))))
 (l(l(l(l))(l(l(l(l)))))(l(l)))(l(l(l(l))(l(l(l(l))))

The λ -calculus

1. Lamdas: instead of $x \mapsto \dots$ we write $\lambda x. \dots$
2. Variable references: for example $\lambda x. x$ denotes the identity.
3. Function applications: instead of $f(x)$ we write fx .

Multi-parameter functions

Problem: no products

$$f : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$$

$$f : (a, b) \mapsto a + b$$

Solution: currying!

$$g : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$$

$$g : a \mapsto (b \mapsto a + b)$$

Note: the two are equivalent in a sense because something something adjoint functors, don't worry about it.

The SKI combinators

$$I := \lambda x. x$$

$$K := \lambda a. \lambda b. a$$

$$S := \lambda f. \lambda a. \lambda c. (fc)(ac)$$

The SKI combinators

- Any term in the λ -calculus can be written using nothing but the SK combinators. For example $I := SKS$.



A combinator to conquer them all

Enter, the ι -combinator!

$$\iota := \lambda x. x \text{ S K} = \text{S}(\text{S I}(\text{K S}))(\text{K K}).$$

We can then define the SKI combinators in terms of ι :

$$\text{I} = \iota \iota$$

$$\text{K} = \iota(\iota(\iota \iota))$$

$$\text{S} = \iota(\iota(\iota(\iota \iota)))$$

Defining $\mathbb{N}$

- Problem: we normally define $\mathbb{N}$ using `zero` and `succ`, but we have neither of those things.
- Solution: ask the person using the numbers to provide them!

Defining $\mathbb{N}$

Example: instead of defining $1 := \text{succ}(\text{zero})$, let

$$1 := \lambda \text{zero} . \lambda \text{succ} . \text{succ } \text{zero} .$$

We can keep going!

$$2 := \lambda \text{zero} . \lambda \text{succ} . \text{succ}(\text{succ } \text{zero})$$
$$3 := \lambda \text{zero} . \lambda \text{succ} . \text{succ}(\text{succ}(\text{succ } \text{zero}))$$
$$4 := \lambda \text{zero} . \lambda \text{succ} . \text{succ}(\text{succ}(\text{succ}(\text{succ } \text{zero})))$$
$$\vdots$$

Working with $\mathbb{N}$

How do we define addition?

1. The function takes two arguments: $\lambda a. \lambda b. \dots$
2. The result is also a natural: $\lambda a. \lambda b. \lambda \text{zero}. \lambda \text{succ}. \dots$
3. Idea – return a , but with its concept of “zero” defined as the natural given by b : $\lambda a. \lambda b. \lambda \text{zero}. \lambda \text{succ}. a (b \text{ zero } \text{succ}) \text{succ}$

$$a + b := \lambda \text{zero}. \lambda \text{succ}. a (b \text{ zero } \text{succ}) \text{succ}.$$

Ordered lists

Ordered lists are surprisingly similar to $\mathbb{N}$. We need:

- The empty list (call it `nil`)
- A way to go from $a_1, a_2, \dots, a_n$ to $x, a_1, a_2, \dots, a_n$ (call it `cons`)

Example: We can write $[1, 2, 3]$ as

```
λ nil. λ cons. cons 1 (cons 2 (cons 3 nil))
```

Note: the naturals are essentially just ordered lists of the unit type, but don't worry about that.

The general principle

- We can encode any inductively defined structure into the λ -calculus this way
- Named after the logician **Alonzo Church**



Encoding text as iotas

- Strings of text are nothing but ordered lists of characters.
- Characters can be represented as naturals (indices inside some alphabet)

Encoding text as iotas

1. Take the input string, and church-encode it:

"meow" →

[illegible]

2. Normalize (evaluate) the output:

[illegible]

Encoding text as iotas

3. Convert to SK combinators

[illegible]

The final result

[illegible]

**But like, isn't this
kinda useless**

...well yeah, the final result kinda is ;-;
...but the individual components aren't!

The silver lining

- The λ -calculus itself is extremely important!
- Many functional programming languages (Haskell, Lean, etc) have it at their very core!

Functional programming mentioned



🌟 Thanks You! 🌟